

CSU23021 – Microprocessor Systems

Assignment #02

Creating a Game to Teach Morse Code

Group 22 CSU23021-GR22

Member	Role
Sankalp	Project Workflow Owner
Jacob	GitLab Workflow Owner
Tashfia	Project Documentation Owner
Jing Yang	Project Demonstration Owner

Module: CSU23021 – Microprocessor Systems
School: School of Computer Science and Statistics
University: Trinity College Dublin
Deadline: 23:59, Friday 11 April 2026

Introduction

For this assignment we had to build a Morse Code learning game that runs on the Raspberry Pi Pico (RP2040, Maker Pi Pico board) and is written using a mix of C and ARM Thumb assembly. The idea behind the game is that a player works through four difficulty levels, progressively moving from individual characters with the Morse pattern shown, all the way up to full words with no hints at all.

The player's only input device is the **GP21** push-button. A short press (≤ 255 ms) registers as a dot (.), while anything longer is a dash (-). Once the player stops pressing for a full second, the system inserts a character space; a second second of silence signals that the sequence is complete and passes it to the game logic for checking.

We split the work cleanly across two source files. The interrupt handler and Morse input buffering live entirely in ARM assembly (`assign02.S`), which was the main requirement. Everything else — the game logic, RGB LED, PWM buzzer, watchdog, and display output — is handled in C (`assign02.c`). A PIO program (`ws2812.pio`) takes care of the timing-sensitive WS2812 NeoPixel protocol.

The key hardware resources we used are:

- **GP21** – push-button (rising and falling edge GPIO interrupts)
- **GP28** – WS2812 NeoPixel RGB LED driven by PIO state machine
- **GP18** – PWM buzzer for audible dot/dash feedback
- **TIMER_ALARM0** – 1-second periodic alarm for silence detection
- **RP2040 Watchdog** – 9-second timeout resets an idle game

Workflow

Team Roles

We split the four ownership roles at the start of week one and tracked everything through GitLab Issues. Code was shared across all four of us, but Sankalp took on a larger slice of implementation because the other three had additional ownership responsibilities on top.

Member	Primary Role	Responsibilities
Sankalp	Project Workflow Owner	Overall organisation and milestone tracking; most of the C and assembly implementation
Jacob	GitLab Workflow Owner	Repository setup, merge request reviews, branch hygiene
Tashfia	Documentation Owner	Report template, Doxygen comments, weekly issue tracking
Jing Yang	Demonstration Owner	90-second demo video, final hardware testing, word-level questions

Table 1: Team role allocation for Group 22

Milestones

We defined five milestones in GitLab at the start of the project, which gave us clear weekly targets and made it easy to spot when things were slipping.

Milestone	Target	Scope
M1	Week 1	Repository setup, role allocation, initial issues created
M2	Week 2	GPIO ISR and alarm ISR working; basic dot/dash detection confirmed
M3	Week 3	Levels 1 & 2 functional; RGB LED lives system; watchdog
M4	Week 4	Levels 3 & 4 (word mode); PWM buzzer; SOS reset
M5	Week 5	Final testing, report, demo video, submission

Table 2: Project milestones tracked in GitLab

Weekly Task Breakdown

Week 1. We started by forking the course template (`csu23021/pico-apps`) into the group namespace rather than creating a blank repo, which kept the CMake config and `pico-sdk` submodule intact. Sankalp created 26 GitLab issues covering every functional requirement, then we divided them up. The `develop` integration branch was set up alongside a naming convention for feature branches.

Week 2. Most of this week went into the interrupt handler. We got `gpio_isr` and `alarm_isr` working in assembly and verified basic dot/dash detection over the serial console. One problem that took a while to track down was a USB CDC reboot loop caused by enabling the watchdog before the host had a chance to enumerate the device — we fixed it by moving watchdog enable to game start.

Week 3. Levels 1 and 2 came together here. We also wired in the RGB LED lives display and added `strip_spaces()` to do lenient Morse matching (so a natural pause mid-character does not count as a wrong answer). A second annoying bug was the `r5` ISR reset — the debounce counter was being reset on every interrupt including bounces, which caused false sequence submissions.

Week 4. We extended the game to Levels 3 and 4 using five words (PICO, CODE, MICRO, INTEL, GROUP) cycling via an index variable. The PWM buzzer and SOS reset were both completed and tested on hardware this week.

Week 5. Integration testing, statistics display, Doxygen comments, and the final report write-up. Jing Yang recorded the demo video after we confirmed all four levels worked end-to-end.

GitLab Repository & Collaboration

Repository Structure

The repository was forked from `gitlab.scss.tcd.ie/csu23021/pico-apps`. We used three branch levels: `main` (protected, only receives merge requests from `develop`), `develop`

(integration branch where all feature work is merged before going to `main`), and individual feature branches named `issue-N-short-description`.

Issues and Branches

We tracked 26 issues in total, each with an assignee, a milestone label, and a corresponding feature branch. Jacob reviewed all merge requests before they went into `develop`, which helped catch a few mistakes early — in particular one where a member pushed directly to `develop` without a branch (we caught it quickly and fixed the history).

Commit Statistics

```

Everything up-to-date
katyyiii@katys-MacBook-Air assignment2-morse-game % git log --oneline
14382ca (HEAD -> feature/level-progression, origin/feature/level-progression) feat: implement level progression
c6eb97c (origin/feature/ws2812-led-functions, feature/ws2812-led-functions) merge: combine lives system with LED branch
19861e9 (feature/lives-system) feat: implement lives system (work in progress)
cc3b6ab fix: rename LED functions to match issue spec
347cf7a Add WS2812 RGB LED control functions and fix build
074fbc6 Add WS2812 RGB LED control functions
36245d3 (origin/main, origin/HEAD, origin/9-implement-morse-code-decoder-loop-table-2, origin/7-write-c-ws2812-rgb-led-control-functions-2, main) Merge branch 'feature/issue-6' into 'main'
3260455 fix: restore README and gitignore (should not be deleted)
83413d7 Implement ASM Morse code input buffer logic
102599a (origin/feature/asm-gpio-interrupt-handler) Merge branch 'chore/cmake-lists-and-folder-structure' into 'main'
9446b7a Add CMakeLists and folder structure
4ca151a chore: initial project setup with README and gitignore
dcc952f Initial commit
katyyiii@katys-MacBook-Air assignment2-morse-game %

```

Figure 1: Git commit history showing contributions from all group members

This commit history shows the contributions and development progress of the team throughout the project.

Member	Key Areas
Sankalp	C game logic, PWM buzzer, SOS reset, ISR debugging
Jacob	Repo management, ASM ISR setup, merge reviews
Tashfia	Documentation, Doxygen, report compilation
Jing Yang	Demo video, final testing, word-level questions

Table 3: Summary of contributions by team members

Code Implementation

Architecture Overview

The application is split across four files. `assign02.c` contains the C layer: PIO and LED initialisation, PWM buzzer, game logic, the Morse lookup table, and all functions that get called from assembly. `assign02.S` is the ARM Thumb assembly file containing the main game loop, the GPIO ISR, the alarm ISR, level selection, and user input dispatch. `ws2812.pio` is the PIO program that handles WS2812 NeoPixel timing, and `CMakeLists.txt` pulls in the required SDK libraries and enables USB CDC stdio.

GPIO Interrupt Handler

`gpio_isr` is installed into the RP2040 RAM vector table at offset `0x74` (GPIO interrupt #13). On every button event it reads `IO_BANK0_PROCO_INTS2` to find the edge type, clears both rise and fall flags via `IO_BANK0_INTR2`, then branches on the result.

For a rising edge (button press), it records the current timestamp in `press_start` by calling `get_time_in_ms()`. For a falling edge (button release), it computes the press duration, checks it against a 50ms debounce threshold, resets `timer_ticks` and `press_start`, re-arms the alarm, and finally writes either 1 (dot, duration < 255 ms) or 2 (dash) to `isr_input`.

One thing worth noting: we use `TST` rather than `CMP` to check individual flag bits. This matters because rise and fall flags can arrive simultaneously in the same interrupt cycle, and `CMP` would give the wrong answer in that case.

A bug we hit early was that the debounce counter was being reset at the top of the ISR unconditionally — on every interrupt, including bounces. This caused bounce events to corrupt the silence counter and trigger premature sequence submissions. The fix was to gate the reset so it only happens on a confirmed valid falling edge.

Alarm Timer ISR

`alarm_isr` fires every 1 second via `TIMER_ALARM0`. Before incrementing anything it checks whether a button press is currently active (`press_start` $\neq$ 0) and whether an unread dot or dash is still pending (`isr_input` $\neq$ 3). If either is true the tick is skipped. Otherwise `timer_ticks` is incremented: value 1 signals a character space and value 2 signals sequence submission.

Main Game Loop

`main_asm` runs the entire game state machine. On restart it calls `gameStart()` to reset lives and counters, then enters level-selection mode where it loops on `wfi + user_input` until the player enters the Morse code for a valid level number (`.--` for 1, `..--` for 2, `...-` for 3, `....-` for 4). It also polls `is_reset_requested()` each iteration so that an SOS input can interrupt level selection.

Once a level is chosen, the gameplay loop feeds the watchdog, waits on `wfi`, calls `user_input`, and then calls `check_level_complete()`. A return value of 1 means the player has answered five in a row correctly and should move up a level. A return value of 2 means game over or an SOS reset — in either case the loop branches back to `restart_game`.

Morse Input Buffer

`detectInput(int input)` in C receives the decoded input code (1 = dot, 2 = dash, 3 = space, 4 = enter) and manages a 32-byte `inputSequence` buffer. When a dot or dash arrives, it appends the character, triggers an immediate buzzer beep, and checks `is_sos_sequence()` in case the player has typed `...--...`. On a space it inserts a separator but guards against consecutive duplicates. On enter it strips trailing spaces, sets `inputComplete = 1`, and calls either `display_user_input()` (gameplay) or `select_level()` (during level selection), depending on context.

The `strip_spaces()` helper is used whenever two Morse strings are compared. This means a natural 1-second pause mid-character — which inserts a space in the buffer — does not cause a wrong answer, since spaces are removed before comparison.

RGB LED Lives System

The LED colour is updated every time a life is gained or lost by calling `RGB_Lives_Display(int remaining_lives)`. We chose colours that are easy to distinguish on the hardware:

Lives	Colour	RGB (hex)
3	Purple	(0x4B, 0x00, 0x82)
2	Orange	(0xFF, 0xA5, 0x00)
1	Red	(0x7F, 0x00, 0x00)
0	Off	(0x00, 0x00, 0x00)

Table 4: RGB LED colour scheme for remaining lives

Blue (0x00, 0x00, 0x7F) is displayed during level selection and after an SOS reset to show the game is not active. Green is set at the start of each new game.

Four-Level Game Structure

Level 1 A random alphanumeric character (A–Z, 0–9) is picked from the 36-entry Morse table, and both the character *and* its Morse code are shown to the player.

Level 2 The same random character selection, but only the character is displayed — no Morse hint.

Level 3 One of five words (PICO, CODE, MICRO, INTEL, GROUP) is presented with its full word-level Morse encoding shown. Questions cycle in order via `level3QuestionIdx`.

Level 4 Same five words in the same order, but with no Morse hint.

All four levels share the same lives rules: a correct answer adds a life (capped at 3), a wrong answer removes one. Five consecutive correct answers in a single level triggers progression; zero lives triggers game over with a statistics summary.

Watchdog Timer

The watchdog is enabled inside `level_select_false()` rather than at boot. We had to do this because enabling it early caused a reboot loop: the watchdog would fire before the

USB host finished enumerating the CDC device, which then rebooted the board, which re-triggered enumeration, and so on. Once gameplay starts, `watchdog_update()` is called on every main loop iteration and inside the alarm ISR. If the player goes idle for roughly 9 seconds, the board resets and the welcome banner re-appears.

WS2812 PIO Program

`ws2812.pio` uses the standard Raspberry Pi WS2812 timing program. We set `IS_RGBW = false` because the on-board LED is a standard three-channel RGB device. The `% c-sdk {}` block initialises the state machine using `ws2812_T1`, `ws2812_T2`, `ws2812_T3` constants generated from the `.define public` directives in the PIO source. This avoids undeclared-identifier errors that can appear if computed expressions are used directly in the SDK init block.

Innovative Features

Feature #1 – PWM Buzzer Dot/Dash Feedback

Morse Code was originally designed to be heard, not seen. Every confirmed dot plays a 150 ms tone and every confirmed dash plays a 450 ms tone through the Maker Pi Pico buzzer on **GP18**, giving the player immediate auditory feedback the moment their input is registered. In practice this turned out to be genuinely useful — you can tell from the sound rhythm alone whether you pressed long enough for a dash, without having to watch the console.

`buzzer_init()` sets GP18 to PWM mode and configures the slice with a clock divider of 125 and a wrap of 499, producing roughly a 2 kHz tone at 50% duty cycle. PWM is left disabled until a beep is needed. `buzzer_beep(int duration_ms)` enables the slice, blocks for the given duration, then disables it. The calls sit inside `detectInput()` immediately after a dot or dash is appended to the buffer, so the sound is synchronous with input registration. The ARM ISR is not modified.

Feature #2 – SOS Emergency Reset

At any point during the game — even mid-sequence, before the 2-second submit timeout — the player can type the international distress signal SOS (`...--...`) using the same GP21 button to immediately reset back to the welcome banner. The feature works during active gameplay and during level selection. On reset the LED switches to blue to signal the idle state.

The motivation was to give players a clean escape without unplugging the board. Using SOS specifically makes the reset itself a Morse learning moment — the player has to know the signal to use it — and it is documented in the welcome banner from the start.

Detection. `is_sos_sequence()` calls `strip_spaces()` on `inputSequence` into a local buffer and then does a `strcmp` against `"...--..."`. Stripping spaces first is the key design choice: when the player pauses between the S and O groups the alarm ISR inserts a space, leaving `"... -- ..."` in the buffer. Stripping makes both forms match identically, reusing the same helper already used for lenient answer checking. The check is called live after every single dot or dash is appended in `detectInput()`, so the reset fires on the

ninth character without needing the 2-second submit timeout. A fallback check on the enter path covers the paused case too.

C-assembly bridge. The shared state is a `volatile int reset_requested` flag. `volatile` is necessary because the flag is written by C and polled by assembly across loop iterations, so the compiler must not optimise it into a register. Two thin wrappers — `is_reset_requested()` and `clear_reset_request()` — expose it to assembly without assembly needing to know its address.

Assembly integration. During gameplay, `check_level_complete()` checks the flag at its top and returns 2, which the main loop's `beq restart_game` branch picks up. During level selection, `level_input` has its own explicit `bl is_reset_requested` poll after each `user_input` call, branching to a dedicated `sos_restart_from_level_select` label that clears the flag and jumps to `restart_game`. The ARM interrupt handler is not modified at any point.

Individual Contributions (D1)

Sankalp – Project Workflow Owner

Sankalp was responsible for the overall project organisation as well as the bulk of the implementation. On the setup side, he forked the course template into the group namespace (not a blank repo, so the CMake config and `pico-sdk` submodule were preserved), created the 26 GitLab issues and 5 milestones, and established the `develop` branch and feature-branch naming convention.

On the code side, his main contributions were across all four source files:

- `CMakeLists.txt`: added the `hardware_pwm` library link and configured USB CDC stdio following the `lab10` pattern.
- `assign02.c`: full boot sequence in `main()` including the 13-second startup delay, PIO setup, and dummy IRQ registration; GPIO helper functions; watchdog init and enable logic; RGB LED lives display; PWM buzzer; Morse input buffer (`detectInput()`, `strip_spaces()`, `initalizeInputArray()`); answer checking (`checkAnswer()`, `match_word()`, `checkMorseCode()`); game state management (`gameStart()`, `select_level()`, `check_level_complete()`).
- **SOS Emergency Reset (Innovative Feature #2)**. Designed and implemented the full SOS reset system: the `volatile int reset_requested` shared flag, `is_sos_sequence()` (space-stripped comparison against "...--..."), `reset_to_welcome()` (console output, LED to blue, welcome banner), and the two assembly bridge wrappers `is_reset_requested()` and `clear_reset_request()`. Also integrated the live detection call after every dot/dash inside `detectInput()`, the fallback check on the enter path, and the assembly polling at both the gameplay loop and the `level_input` subroutine — so SOS works regardless of what state the game is in when it is triggered.
- `assign02.S`: main game loop (`main_asm`, `restart_game`, `level_not_chosen`); GPIO ISR (`gpio_isr`) including TST-based flag checking and debounce; alarm ISR (`alarm_isr`); ISR vector table installation; input dispatch (`user_input`); level selection subroutine (`level_input`).

Key bugs he debugged included the USB CDC watchdog reboot loop, the GPIO pull-up

loss after `gpio_init()`, the SDK IRQ assertion requiring the dummy callback, and the `r5` ISR reset bug.

Jacob – GitLab Workflow Owner

Jacob managed the repository end-to-end: reviewing and merging all feature branch merge requests into `develop`, enforcing the branch naming convention, and keeping the commit history clean and descriptive. On the code side, he contributed to the ARM assembly ISR subroutines — specifically the vector table patching and NVIC register writes in `install_gpio_isr` and `install_alm_isr`, and helped with the alarm re-arming logic in `set_alarm`.

Tashfia – Project Documentation Owner

Tashfia set up the initial report template and structure, compiled each member’s individual contribution sections into this unified document, and added Doxygen comment blocks to all public C functions in `assign02.c`. She also tracked weekly task completion in the GitLab issue tracker and kept the milestone statuses up to date.

For code she contributed the welcome banner text and the level-selection display messages (`display_level_choosing()`, `level_selection_invalid()`), and helped populate the Morse code lookup table (`morse_init()`) and the `que[]` and `alp[]` static arrays.

Jing Yang – Project Demonstration Owner

Jing Yang was responsible for preparing and recording the final hardware demonstration video for the project. She made sure the demo clearly showed the main features of the game, including Morse code input detection, level selection, level progression, life reduction, LED colour feedback, and the word-based higher levels.

She also carried out final hardware testing before recording the demonstration to make sure the system was working correctly across all four levels and that the user experience was smooth during gameplay.

Her coding contribution focused on the word-based part of the game used in the higher difficulty levels. She worked on the functions `level_3_question()` and `level_4_question()`, which handle the word-level questions shown to the player. This included the logic for cycling through the five words PICO, CODE, MICRO, INTEL, and GROUP using the question index variables.

She also contributed to the word-matching logic in `display_user_input()` for the word-level cases, helping ensure that the player’s Morse input was checked correctly for Levels 3 and 4.

Conclusion

We came out of this project with a fully working four-level Morse Code game. All required features are in place: level progression, the three-lives system with RGB LED feedback, Morse input detection through GPIO interrupts in ARM assembly, and a watchdog-triggered reset for idle games. Both innovative features also made it into the final build —

PWM buzzer feedback that makes the dot/dash rhythm audible, and the SOS emergency reset that gives players a Morse-native way to restart without touching the hardware.

The trickiest part technically was getting the ISR right. The combination of simultaneous rise/fall flags, bounce events, and the USB CDC enumeration timing meant we had several debugging sessions that ran longer than expected. Working through those with git history and serial logging helped a lot.

From a process perspective, the GitLab Issues and milestone structure was genuinely useful — it kept everyone on the same page and made it easy to see at a glance what was blocking progress during weekly check-ins.